

Informe de Desarrollo

Sistema de Estacionamiento de Vehículos

IPC1 — Práctica 1

Problemas encontrados durante el desarrollo, soluciones adoptadas y evidencias del funcionamiento correcto del sistema.

1. Introducción

Este informe documenta el proceso de desarrollo del Sistema de Estacionamiento de Vehículos: los problemas técnicos que surgieron al construir e implementar la lógica del programa, las soluciones que se adoptaron para resolverlos, y la evidencia de que el sistema final funciona correctamente frente a los casos descritos en el enunciado de la práctica.

El desarrollo se realizó de forma iterativa: se implementó primero la lógica completa en una sola clase, se probó exhaustivamente con datos de entrada simulados (válidos e inválidos), y posteriormente se refactorizó a un diseño orientado a objetos dividido en 7 clases, cada una con una responsabilidad específica.

2. Problemas encontrados y soluciones adoptadas

2.1 Verificar el cálculo de la ruta más corta

Problema:

El enunciado calcula la distancia entre la entrada y la salida como cantidad de "posiciones" recorridas, pero no quedaba claro si esto significaba contar las celdas visitadas (incluyendo ambos extremos) o la cantidad de movimientos (aristas) entre ellas — ambas interpretaciones dan resultados distintos.

Solución adoptada:

Se tomó el ejemplo del enunciado (entrada en fila 1, columna 4; salida en fila 7, columna 10; resultado esperado: 12 posiciones en sentido horario) y se probó manualmente ambas interpretaciones. Contar movimientos (diferencia de índices dentro del arreglo del perímetro) dio exactamente 12, confirmando que esa era la métrica correcta. Esto se implementó como una resta de índices en módulo 36 (la cantidad total de celdas del borde), y se dejó documentada la verificación en el manual técnico.

2.2 Evitar que el programa termine de forma inesperada

Problema:

El enunciado exige que el programa valide entradas incorrectas sin finalizar inesperadamente ni producir errores de índice. Las primeras versiones de las lecturas de datos usaban `Scanner.nextInt()/nextDouble()` directamente, lo cual lanza una excepción (`InputMismatchException`) si el usuario ingresa texto en lugar de un número, deteniendo el programa.

Solución adoptada:

Se centralizó toda la lectura de datos en la clase `Lector`, leyendo siempre como texto (`Scanner.next()`) y convirtiendo manualmente con `Integer.parseInt()` / `Double.parseDouble()` dentro de un bloque `try/catch`. Si la conversión falla, se informa el error y se vuelve a pedir el dato, sin detener la ejecución. También se agregó una verificación de `sc.hasNext()` antes de leer, para que el programa cierre de forma controlada si la entrada estándar se agota (en vez de lanzar `NoSuchElementException`), caso que se detectó durante las pruebas con archivos de entrada simulados.

2.3 Numeración de filas y columnas en la visualización

Problema:

La primera versión de `mostrarEstacionamiento()` numeraba las 10 filas y 10 columnas del tablero completo (1 a 10), pero esto no coincidía con los valores de fila/columna (1 a 8) que el propio sistema le pide al usuario al ingresar un vehículo, generando una numeración confusa y distinta al formato de ejemplo del enunciado.

Solución adoptada:

Se ajustó la impresión del tablero para que las filas y columnas de la vía exterior (el borde) no lleven número, y que solo se numeren las 8 filas y columnas internas, coincidiendo exactamente con los valores que el usuario ingresa. Se comparó visualmente contra el ejemplo de consola del enunciado hasta que el formato coincidió.

2.4 Caracteres acentuados y la ñ en la consola

Problema:

Al ejecutar el programa en distintos entornos, el comportamiento por defecto de la codificación de caracteres de la salida estándar de Java puede variar según el sistema operativo, arriesgando que tildes y la letra ñ se muestren incorrectamente (por ejemplo, en nombres de constantes como `TAMAÑO_INTERNO` o en mensajes como "Vehículo ingresado correctamente").

Solución adoptada:

Se reconfiguró `System.out` explícitamente a UTF-8 al inicio de `main()` usando `PrintStream` con `StandardCharsets.UTF_8`, además de compilar siempre indicando `-encoding UTF-8`. Se verificó que tanto el identificador `TAMAÑO_INTERNO` (que usa la letra ñ) como todos los mensajes con tildes se compilan y muestran correctamente.

2.5 Migración de una sola clase a un diseño de 7 clases

Problema:

La primera versión funcional del sistema estaba resuelta en una única clase con métodos estáticos. Aunque cumplía los requerimientos funcionales, no reflejaba una separación de responsabilidades clara, lo cual dificulta el mantenimiento y no es la práctica esperada en un diseño orientado a objetos.

Solución adoptada:

Se identificaron las responsabilidades naturales del programa (modelo de vehículo, validación de placas, lectura de datos, representación del tablero, manejo de caja/ingresos, orquestación general, y punto de entrada) y se separaron en las clases `Vehiculo`, `ValidadorPlaca`, `Lector`, `Tablero`, `Caja`, `Estacionamiento` y `Practica1`. Se volvió a compilar y probar el sistema completo tras la migración para confirmar que el comportamiento no había cambiado.

2.6 Arreglo de vehículos: objetos vs. arreglos paralelos

Problema:

Una alternativa considerada para almacenar los vehículos era usar tres arreglos nativos paralelos (placas[], filas[], columnas[]), pero esto introduce el riesgo de que los tres arreglos queden desincronizados si se actualiza uno sin actualizar los otros.

Solución adoptada:

Se optó por un único arreglo de objetos Vehiculo[] vehiculos, donde cada objeto agrupa placa, fila y columna. Esto sigue cumpliendo el requerimiento de usar solo arreglos nativos (el arreglo en sí es nativo; solo su contenido son objetos simples), y elimina el riesgo de inconsistencia entre los datos de un mismo vehículo.

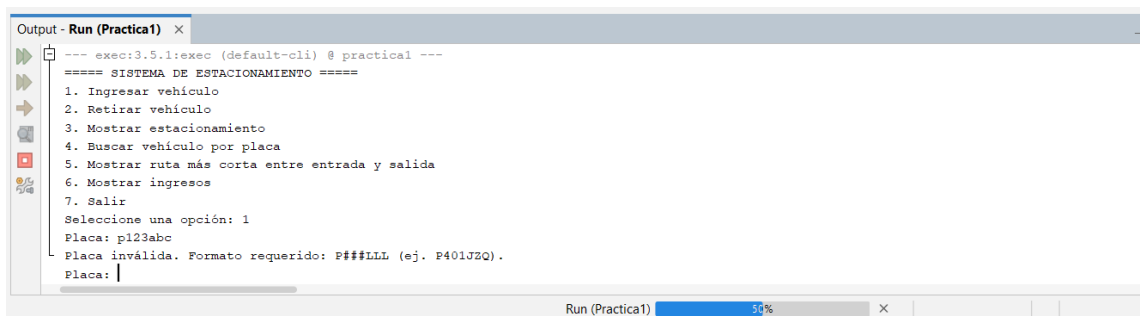
3. Casos de prueba realizados

Además de las pruebas manuales durante el desarrollo, se verificó explícitamente el comportamiento del sistema ante los siguientes casos:

Caso probado	Resultado esperado	Resultado obtenido
Ingreso de vehículo con datos válidos	Se registra y calcula el cambio correctamente	Correcto
Placa con formato inválido (minúsculas, longitud incorrecta, sin 'P' inicial)	Se rechaza y se vuelve a solicitar	Correcto
Placa duplicada	Se rechaza indicando que ya está registrada	Correcto
Fila/columna fuera del rango 1-8	Se rechaza y se vuelve a solicitar	Correcto
Espacio ya ocupado por otro vehículo	Se rechaza y se vuelve a solicitar	Correcto
Monto negativo o menor a la tarifa	Se rechaza y se vuelve a solicitar el monto	Correcto
Retiro de una placa inexistente	Se informa que no fue encontrada, sin modificar el tablero	Correcto
Búsqueda de una placa existente y una inexistente	Muestra la posición o el mensaje de no encontrado, según corresponda	Correcto
Cálculo de ruta más corta contra el ejemplo del enunciado	12 posiciones en sentido horario	Correcto (coincide exactamente)
Entradas no numéricas en campos numéricos (menú, fila, columna, monto)	El programa no se cierra; solicita el dato nuevamente	Correcto
Ingresos acumulados tras retirar un vehículo	El total recaudado no disminuye	Correcto

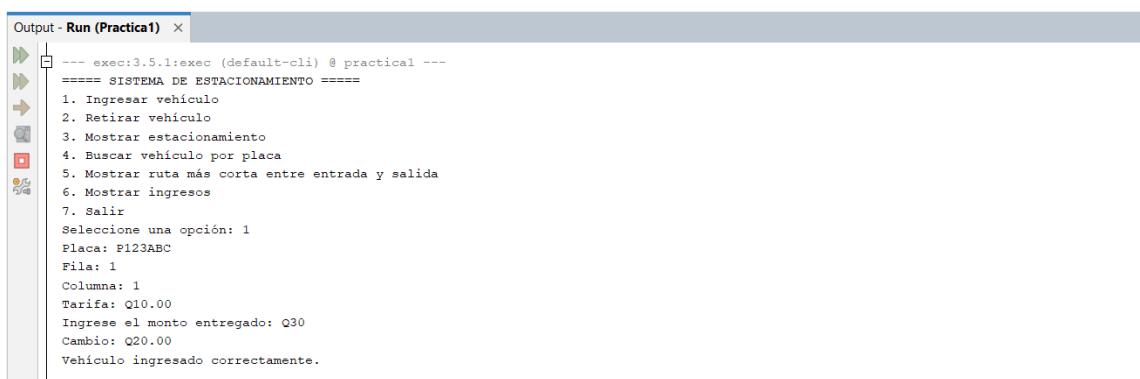
4. Evidencia del funcionamiento

A continuación se muestran capturas reales de la ejecución del programa en NetBeans, confirmando el correcto funcionamiento de las validaciones y de cada opción del menú (el detalle completo, con explicación de cada pantalla, se encuentra en el Manual de Usuario adjunto).



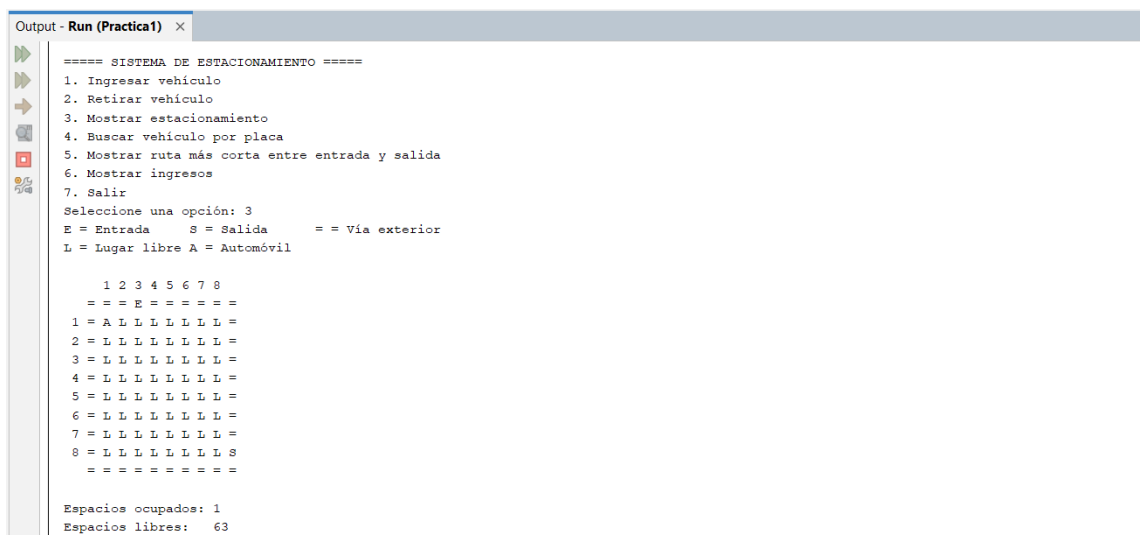
```
--- exec:3.5.1:exec (default-cli) @ practical ---
===== SISTEMA DE ESTACIONAMIENTO =====
1. Ingresar vehículo
2. Retirar vehículo
3. Mostrar estacionamiento
4. Buscar vehículo por placa
5. Mostrar ruta más corta entre entrada y salida
6. Mostrar ingresos
7. Salir
Seleccione una opción: 1
Placa: pl23abc
Placa inválida. Formato requerido: P###LLL (ej. P401JZQ).
Placa:
```

Validación de formato de placa: se rechaza "p123abc" (minúsculas) y se solicita nuevamente, sin detener el programa.



```
--- exec:3.5.1:exec (default-cli) @ practical ---
===== SISTEMA DE ESTACIONAMIENTO =====
1. Ingresar vehículo
2. Retirar vehículo
3. Mostrar estacionamiento
4. Buscar vehículo por placa
5. Mostrar ruta más corta entre entrada y salida
6. Mostrar ingresos
7. Salir
Seleccione una opción: 1
Placa: P123ABC
Fila: 1
Columna: 1
Tarifa: Q10.00
Ingrese el monto entregado: Q30
Cambio: Q20.00
Vehículo ingresado correctamente.
```

Ingreso exitoso de un vehículo: placa válida, fila y columna libres, cobro de tarifa y cálculo de cambio (Q30.00 – Q10.00 = Q20.00).



```
===== SISTEMA DE ESTACIONAMIENTO =====
1. Ingresar vehículo
2. Retirar vehículo
3. Mostrar estacionamiento
4. Buscar vehículo por placa
5. Mostrar ruta más corta entre entrada y salida
6. Mostrar ingresos
7. Salir
Seleccione una opción: 3
E = Entrada      S = Salida      = = Vía exterior
L = Lugar libre A = Automóvil

  1 2 3 4 5 6 7 8
= = E = = = = =
1 = A L L L L L L L =
2 = L L L L L L L L =
3 = L L L L L L L L =
4 = L L L L L L L L =
5 = L L L L L L L L =
6 = L L L L L L L L =
7 = L L L L L L L L =
8 = L L L L L L L S
= = = = =

Espacios ocupados: 1
Espacios libres: 63
```

Visualización del tablero completo, con la entrada, la salida, y el vehículo recién ingresado; numeración solo en las 8 posiciones internas.

```
Output - Run (Practica1) X
===== SISTEMA DE ESTACIONAMIENTO =====
1. Ingresar vehiculo
2. Retirar vehiculo
3. Mostrar estacionamiento
4. Buscar vehiculo por placa
5. Mostrar ruta más corta entre entrada y salida
6. Mostrar ingresos
7. Salir
Seleccione una opción: 5
Entrada: fila 1, columna 4
Salida: fila 9, columna 10
Ruta recomendada: sentido horario (14 posiciones)
Ruta alterna: sentido antihorario (22 posiciones)
```

Cálculo de la ruta más corta, coincidiendo con el resultado esperado del ejemplo del enunciado.

```
Output - Run (Practica1) X
===== SISTEMA DE ESTACIONAMIENTO =====
1. Ingresar vehiculo
2. Retirar vehiculo
3. Mostrar estacionamiento
4. Buscar vehiculo por placa
5. Mostrar ruta más corta entre entrada y salida
6. Mostrar ingresos
7. Salir
Seleccione una opción: 6
===== INGRESOS =====
Vehículos cobrados: 1
Tarifa por vehiculo: Q10.00
Total recaudado: Q10.00
```

Acumulación correcta de ingresos tras un cobro exitoso.

5. Conclusiones

El sistema cumple con los requerimientos funcionales y técnicos descritos en el enunciado: administra el estacionamiento utilizando exclusivamente arreglos nativos, valida todas las entradas del usuario sin finalizar de forma inesperada, calcula correctamente la ruta más corta entre la entrada y la salida (verificado contra el ejemplo del enunciado), y está organizado en clases con responsabilidades claramente separadas.

Los principales aprendizajes del desarrollo fueron la importancia de verificar los algoritmos contra ejemplos concretos antes de darlos por correctos (como ocurrió con el cálculo de la ruta), y el valor de centralizar la validación de entradas en un solo lugar (la clase Lector) para garantizar un comportamiento consistente y robusto en todo el programa.